



Before You Start — Project Prerequisites

This mini project demonstrates a real-world **Enterprise Agentic AI workflow system** using the following:

- Python
- Streamlit
- LangGraph
- Gemini / Ollama
- MongoDB Atlas Vector Search
- RAG (Retrieval-Augmented Generation)

The application allows users to:

- upload enterprise documents
- perform semantic AI search
- run multi-agent workflows
- generate business insights
- simulate enterprise AI copilots



Required Setup Checklist

Before running this project, ensure the following components are properly configured.

1 Python Environment

Install:

```
Python 3.10+
```

Verify installation:

```
python --version
```

2 Install Required Dependencies

Run:

```
pip install -r requirements.txt
```

Core packages include:

- streamlit
- pymongo
- langgraph
- google-genai
- sentence-transformers
- pypdf
- pandas

3 MongoDB Atlas Setup

This project uses MongoDB Atlas for:

- document storage
- vector embeddings
- semantic search

Create MongoDB Atlas account.

Create a free account:

<https://www.mongodb.com/cloud/atlas>

Create Cluster

Create:

- Free Tier Cluster (M0)

Create Database

Example:

```
Database Name: rag_db  
Collection Name: documents
```

 **IMPORTANT:**
Create a **Standard Collection**

DO NOT choose:

- Time Series Collection
- Capped Collection

4 Create MongoDB Vector Search Index

Go to:

```
Atlas Search  
→ Create Search Index  
→ JSON Editor
```

Paste:

```
{  
  "fields": [  
    {  
      "type": "vector",  
      "path": "embedding",  
      "numDimensions": 768,  
      "similarity": "cosine"  
    }  
  ]  
}
```

Important Notes

Setting	Value
Index Name	vector_index
Field Path	embedding
Embedding Size	768

Wait until index status becomes

ACTIVE

5 Gemini API Setup

This project uses Gemini for:

- embeddings
- AI reasoning
- business summaries

Create Gemini API key.

Open:

<https://aistudio.google.com/>

Generate API key.

Create `.env` File

Example:

```
GEMINI_API_KEY=your_api_key  
  
MONGO_URI=your_mongodb_connection_string
```

6 Streamlit Setup

Run application:

```
streamlit run app.py
```

The application opens in the browser automatically.

7 Upload Supported Files

The project supports:

File Type	Purpose
PDF	Enterprise policies
CSV	Business datasets
XLSX	Financial/HR reports
TXT	Reference documents

8 What Happens Internally

When user uploads a document:

```
graph TD; A[Upload File] --> B[Extract Text]; B --> C[Generate Gemini Embeddings]; C --> D[Store in MongoDB Atlas]; D --> E[Create Vector Search]; E --> F[Retrieve Context]; F --> G[Run Multi-Agent Workflow]; G --> H[Generate AI Business Insights];
```

9 Example Use Cases

This mini project supports:

- Invoice analysis
- Retail analytics
- HR insights

- Operations monitoring
- Policy retrieval
- AI copilots
- Enterprise document search

Important Troubleshooting Notes

If RAG Context Appears Empty

Check:

Problem	Solution
Wrong collection type	Use Standard Collection
Wrong vector dimensions	Use 768 for Gemini
Index not ACTIVE	Wait until ACTIVE
Empty MongoDB collection	Re-upload documents
Wrong index name	Use vector_index

Expected Final Output

The workflow should display:

Plan

1. Analyze uploaded dataset
2. Retrieve relevant policies
3. Detect anomalies
4. Generate executive summary

Context (RAG)

Invoices overdue beyond 15 days require escalation.

Final Output

Business Summary Report

Total delayed invoices: 17

Pending amount: ₹1,82,500

Recommendation:

Escalate invoices delayed beyond 15 days.
